

ML-23/24-09 Approve Prediction of Multisequence Learning

MD Fazley Rabbe (1502895)
fazley.rabbe@stud.fra-uas.de

Mahidul Islam Rana (1502217)
mahidul.rana@stud.fra-uas.de

Abstract— This project presents an innovative Hierarchical Temporal Memory (HTM)-based solution for learning and predicting sequences composed of numeric and character data. By leveraging the Spatial Pooler (SP) and Temporal Memory (TM) algorithms from the NeoCortex API, our system processes user-defined JSON datasets through a scalar encoder that transforms raw data into Sparse Distributed Representations (SDRs). The SP extracts stable, recurring patterns while the TM captures temporal dependencies, enabling robust next-step predictions. The modular architecture, featuring specialized classes for sequence ingestion, learning management, and performance monitoring, supports a range of applications—from cancer prediction to power consumption forecasting. Our results demonstrate that multiple passes of SP-TM training significantly enhance predictive accuracy, underscoring the potential of HTM's biologically inspired approach for solving complex, real-world sequence prediction challenges.

Keywords—Hierarchical Temporal Memory (HTM), Prediction code, Local Area Density, SDR, AI, HTM Prediction Engine).

INTRODUCTION

Multi-sequence learning in Hierarchical Temporal Memory (HTM) is concerned with making a computational model learn and predict patterns from many sequences of time-series data. In this paper, we present a new extension to the HTM pipeline that directly reads training sequences from JSON files, making it easy to ingest large-scale data. The data is encoded into Sparse Distributed Representations (SDRs) using a scalar encoder, which preserves both Character and numerical range, but also the temporal sequence ordering of every sequence. The SDRs are passed to the Spatial Pooler (SP) for learning stable representations of input patterns, which are passed to the Temporal Memory (TM) for encoding longer-term sequential dependencies [1]. After the model finishes its repeated training process, it will automatically start loading test sequences from separate datasets. This setup lets the system predict the next element in each sequence and calculate an overall accuracy. We evaluate this multi-sequence approach in two different fields - cancer prediction and power consumption prediction with HTM showing the patterns and trends within each dataset. By combining an HTM-based process with simple JSON data

handling, the project provides a strong and easy-to-use platform for sequence modeling in many different areas [1].

LITERATURE SURVEY

A. Multisequence Learning

Multisequence learning is a subfield of machine learning that is committed to the joint study and modeling of various sequences of information. It can cover the whole variety of modalities of data sequences time-series data (stock markets or sensor readings, for example), string inputs (words or documents, for instance), biological sequences (DNA or proteins, for example), and other kinds of sequential inputs. The main objective of multisequence learning is to model intra-sequence dependencies (relationships within a given sequence) and inter-sequence dependencies (the way sequences can interact or complement each other).

By analyzing multiple sequences simultaneously, models can capture latent high-order correlations that would not be apparent when each sequence is considered in isolation. In time-series forecasting, for example, it can help to see various but perhaps related streams (e.g., temperature and humidity readings). In text processing, parallel sequences (e.g., bilingual text corpora) can be used to aid tasks like translation or classification. At the same time, biological studies are frequently advantaged by considering several genetic or protein sequences together, using common patterns of structure to enhance classification or anomaly detection [2].

Certain deep learning models are particularly suited to multisequence learning, such as Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and Transformer-based models. These models can all be combined with attention mechanisms, which allow the model to concentrate on the most important section of a sequence or multiple sequences when making a prediction. Hybrid models that combine recurrence and attention are also gaining popularity, as they are better able to capture both local and long-distance dependencies [2].

Practically, the use of multisequence learning in C# (and other languages) is usually through frameworks that provide packages for building sequence models and GPGPU acceleration. Frameworks like ML.NET or TensorFlow and PyTorch bindings may provide special modules to process sequential data input, organize training pipelines, and simplify model deployment. These features allow integration

of multisequence learning in production systems for prediction, classification, clustering, and content creation with the potential advantage of being able to capture cross-sequence dependencies.

B. Hierarchical Temporal Memory

Hierarchical Temporal Memory (HTM) is a biologically inspired approach founded upon the structure and organizational framework of the neocortex with the goal of replicating the manner in which the human brain processes various forms of sensory input—e.g., vision, audition, and behavior—and ultimately generates memory and predictions. Rather than relying on pre-programmed rules, HTM networks learn by being exposed to data and can therefore be put into a wide variety of applications relating to the modeling of complex sensory streams [3]. In HTM, the neurons are arranged in columns inside hierarchically connected regions so that the system can learn spatial and temporal patterns in the arriving data. The basic principles rely on two algorithms, which have been thoroughly explained in the original whitepaper [4]. The network matches each new pattern to those that it has previously viewed within only a few training iterations. To enable the effective prediction of future elements, it is critical that input patterns are adequately distinct, thus enabling HTM to form strong representations [2].

C. Sparse Distributed Representation

One of the fundamental challenges of artificial intelligence is devising an effective method of knowledge representation. Most methods in mainstream computer science are inadequate for representing common sense notions and their interrelations in a form that is readily processable by computers. This difficulty stems from the fact that our understanding of the world is not organized into discrete, easily separable facts but rather as a complex network of interrelated ideas. Conventional data formats fall short in capturing these intricate and often ambiguous connections between concepts. In contrast, the human brain utilizes a method called Sparse Distributed Representation (SDR) to encode information. SDRs play a significant role in biological intelligence and serve as a basis both to represent information and to compute in the brain. In practice, SDRs act as the brain's language, allowing for the flexible and creative handling of data. There are a large number of bits in an SDR but few of them at a time enabled (set to 1), with all the remaining bits in disabled (set to 0) state. All bits in the encoding exist to represent information, as do those in the brain—where an active bit corresponds to an active neuron, and an inactive bit represents a neuron that is less engaged. This sparse coding mechanism not only provides robustness against noise but also allows for the efficient representation of complex patterns, making it a highly desirable approach for artificial systems that aim to emulate human-like intelligence [5].

D. Neocortex

The neocortex is a critical region of the brain used in higher-level cognitive functions like sophisticated reasoning, sensory perception, and intricate motor control. The neocortex possesses a differentiated structure with lots of layers of differentiated neural circuits and specialized types of neurons. This layered organization enables different regions of the neocortex to specialize in various tasks—for instance, some layers are more involved in processing visual or auditory information, while others play a key role in planning and executing complex movements. Inspired by this intricate biological structure, Hierarchical Temporal Memory (HTM) models try to replicate the structural and functional characteristics of the neocortex. HTM systems are organized in hierarchically structured layers that mirror the way the neocortex processes temporal and spatial information. This architecture allows HTM to learn perpetually from data streams, recognizing patterns over time and making predictions about future inputs. This kind of strategy has proven useful in applications like anomaly detection and sequence prediction, reflecting the dynamic learning observed in the human brain [6].

E. Memory

Memory is often viewed as the brain's system for handling information both in terms of conscious (explicit) and unconscious (implicit) processing. It comprises distinct components such as a sensory processor, short-term (or working) memory, and long-term memory. This idea is connected to how neurons work, suggesting a complicated interaction between neurobiology and memory processes [5].

F. Prediction

The C# Neocortex is a smart monitoring system that learns your application behavior during runtime to forecast future issues and optimization potential. Instead of simply reacting to errors, it continuously observes code execution, data flows, and performance patterns to build a dynamic model of your application. By analyzing historical performance data and correlating it with specific code paths, the C# Neocortex can detect potential bottlenecks and predict anomalies before they occur. Predictive insight means that issues can be addressed in advance and coded for optimally with improved performance. In addition, through interactive feedback and acquired experience, the system can contribute to behavior prediction by user and guiding intelligent design and allowing for applications to be flexible and resilient [7].

METHODOLOGY

The multi-sequence learning project, implemented with Microsoft Visual Studio 2022, does an automated reading of training and test sequences from single JSON files to utilize an HTM prediction engine for sequence learning. It uses a Scalar Encoder to convert input data to sparse representations, a Spatial Pooler to learn static patterns which are trained, and then a Temporal Memory module to learn sequential transitions, with a classifier mapping these states to specific predictions. The system processes test sequences character by character, contrasting the forthcoming predicted

element with the actual one, calculating prediction accuracy, and maintaining detailed results in a report, thus demonstrating HTM's capacity for modeling and predicting complex sequential data.

Actually, the prediction works like at inference the Predictor wrapper carries out a forward pass of the trained HTM pipeline but without learning; the current integer is first converted into a sparse distributed representation by the scalar encoder. That SDR is mapped by the spatial pooler onto a stable subset of active columns, and the temporal-memory module using distal synapses learned at training marks out the cells predicted to become active one step on this set of predictive cells is then passed to the lightweight HTM classifier. Which consults the lookup table it built at training and returns the label string most closely matching the current cellular pattern the code then parses the final token of that label to produce the predicted next integer, so the predicted value is simply the one which historically followed the same neural context most often in the training data.

Figure 1 shows the flow chart how this complete experiment and accuracy is calculated

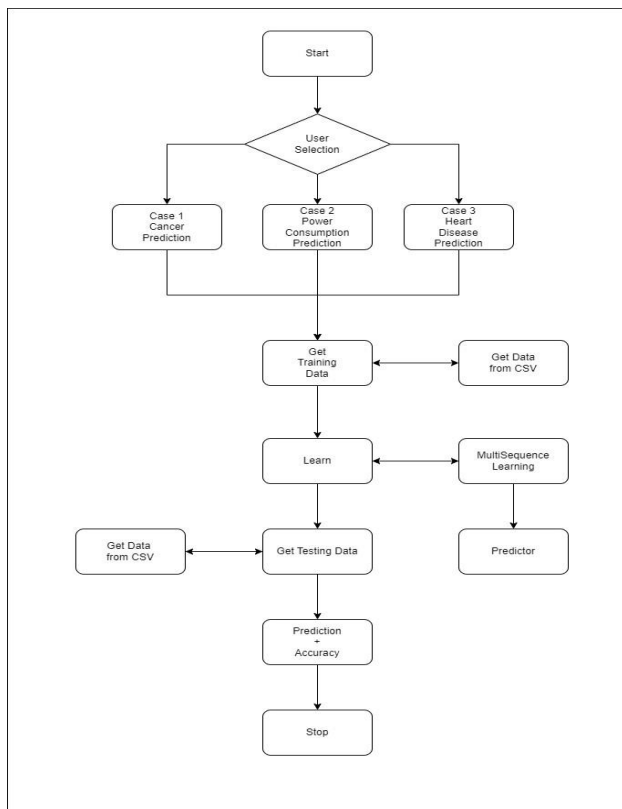


Fig.1: Flow chart for Multi Sequence Learning Experiment

A. Datasets

Our dataset is an organized list of sequential data in the form of JSON, and every entry consists of a labeled sequence of digits or characters compiled by systematic study and observation. Each sequence is framed to detect underlying

trends and patterns, and they form the groundwork for our input to the Multi Sequence Learning project. This data set is not just filled with raw numerical or categorical information but also holds this data in an order that corresponds to real-world phenomena, enabling the HTM predictive engine to learn and predict future items based on historical patterns. By providing a steady and well-classified sequence of data sets, our data set is instrumental in training the Spatial Pooler and Temporal Memory mechanisms, which eventually result in streamlined analysis and accurate predictive output.

Multiple sequence of numbers and alphabet in JSON file for two different scenarios. Figure 2-5 shows the testing and training data sets for both scenarios.

Sequence 1	0	5	6	7	8	10	
Sequence 2	4	6	11	12	13		
Sequence 3	8	1	2	3	4		
Sequence 4	2	3	4	7	8	10	11

Fig.2: Training dataset for power consumption scenario.

Sequence 1	E	F	G	H	I	J	K	L	M	
Sequence 2	P	Q	R	S						
Sequence 3	A	B	C	D	E	F	G	H	I	J
Sequence 4	C	D	G	H	J	K	L	M		

Fig.3: Training dataset for cancer prediction scenario.

Test 1	5	6	7	8	
Test 1	6	11	12	13	
Test 1	1	2	3	4	
Test 1	3	4	7	8	10

Fig.4: Test dataset for power consumption scenario.

Test 1	E	F	G	H
Test 2	P	Q	R	S
Test 3	A	B	C	D
Test 4	C	D	G	H

Fig.5: Test dataset for cancer prediction scenario.

B. Encoder

In HTM systems, the role of the encoder is to transform raw input data numeric values, categorical data, or otherwise into Sparse Distributed Representations (SDRs), which are high-dimensional binary vectors with a small percentage of active (1) bits out of a large number of inactive (0) bits. This transformation is valuable as the HTM network processes SDRs exclusively, not merely normalizing the input form but also semantic relations encoding: similar inputs in the semantic sense result in SDRs that are highly overlapping in their active bits, making the network pattern detection and prediction feasible from partial similarity. The scalar encoder, which is the primary encoder of the HTM system, accomplishes this by mapping continuous scalar inputs into a fixed-length binary vector through a process of dividing the input range into overlapping buckets, with each bucket setting a specific set of bits. This method guarantees determinism any identical input will always produce the same SDR and provides a consistent dimensionality for all encoded outputs, which is important for effective pattern matching and learning within the HTM architecture. In addition, parameters like the total number of bits, the number of active bits, and overlap between successive buckets need to be fine-tuned in a very subtle balance between sensitivity and generalization so that the network can discriminate strongly between close but different inputs and, at the same time, recognize underlying trends and sequences.

C. Spatiol pooler

The Spatial Pooler (SP) is the first processing layer in the HTM network, tasked with converting the high-dimensional, sparse binary output from the encoder into a consistent set of active columns that effectively represent the input's features. It accomplishes this by computing an overlap score for each column—measuring how well the column's synaptic connections match the active bits in the SDR—and then applying a competitive inhibition mechanism to select only a fixed percentage of columns to be active, thereby enforcing a predetermined sparsity level. This dual role of preserving the semantic similarity inherent in the input (so that similar inputs produce overlapping active columns) and maintaining uniform sparsity is analogous to normalization in traditional neural networks, where controlling neuronal activity is key to stabilizing the training process. Additionally, the SP is implemented in three versions to meet different computational demands: a single-threaded version for basic, sequential processing; SP-MT, which harnesses multithreading on a single system to improve efficiency; and SP-Parallel, which leverages both multicore and multimode processing to offer enhanced scalability and performance in more complex environments.

D. Temporal Memory

Hierarchical Temporal Memory (HTM) emulates neocortical processing by first converting raw input data into Sparse Distributed Representations (SDRs) using an encoder, typically a scalar encoder, which maps similar inputs to overlapping high-dimensional binary vectors. These SDRs

are then fed into the Spatial Pooler, the network's initial layer, where each column computes an overlap score based on its synaptic connections to the active bits in the SDR. Through a competitive inhibition mechanism, only a fixed percentage of columns become active, ensuring consistent sparsity and effectively normalizing the input while preserving its semantic features. Building on this spatial representation, the Temporal Memory module captures the dynamic, sequential aspects of the data by forming hierarchical connections across multiple dendritic segments. This allows neurons to learn sequences, enter predictive states when recognizing familiar patterns, and handle multiple potential future outcomes simultaneously. By integrating these spatial and temporal processes, HTM not only generates accurate predictions based on historical patterns but also adapts to new data, making it particularly effective for applications like natural language processing, anomaly detection, and sensor data analysis.

Parameters	Power Consumption	Cancer prediction
inputBits	100	100
numColumns	1024	1024
CellsPerColumn	32	25
GlobalInhibition	false	true
LocalAreaDensity	0.03	-1
NumActiveColumns PerInhArea	0.02 * numColumns	0.02 * numColumns
PotentialRadius	0.5 * inputBits	0.15 * inputBits
MaxBoost	10.0	10.0
DutyCyclePeriod	25	25
MinPctOverlapDutyCycles	0.75	0.75
MaxSynapsesPerSegment	0.02 * numColumns	0.02 * numColumns
Activation Threshold	10	15
Connected Permanence	0.2	0.5
Permanence Decrement	0.01	0.25
Permanence Increment	0.03	0.15
Predicted Segment Decrement	0.005	0.1

Table.1: HTM Config parameters

This table shows the key configuration parameters for the Hierarchical Temporal Memory (HTM) model used in our project. Each parameter such as the number of cells per column, the permanence thresholds, or the level of inhibition helps determine how the Spatial Pooler and Temporal Memory learn and make predictions. For example, “CellsPerColumn” controls how many cells exist in each column for representing different contexts, “ConnectedPermanence” defines the threshold above which a synapse is considered connected, and “PotentialRadius” influences how widely columns can sample input bits. By tuning these values, we balance the HTM’s learning speed, predictive accuracy, and computational efficiency for our specific use case.

E. How the project Works

The project begins by initializing the system and allowing the user to choose between two prediction cases such as cancer prediction or power consumption prediction after which it loads the training data from JSON files containing carefully structured sequences (numeric or character-based) that are kept small to minimize execution time. Once ingested, the raw data is passed through a Scalar Encoder, which converts it into Sparse Distributed Representations (SDRs) while preserving the inherent semantic similarities and temporal ordering of the input. These SDRs then feed into the Spatial Pooler, where the system identifies and extracts stable, active columns using a competitive inhibition mechanism that ensures fixed sparsity, analogous to normalization in traditional neural networks. Building on this spatial representation, the Temporal Memory module learns the sequential dependencies over time by forming hierarchical connections between neurons, thereby enabling the model to predict future elements based on past patterns. After multiple training cycles, the model automatically transitions to testing mode by loading separate test sequences from JSON files, where the Predictor module forecasts the next elements in each sequence. These predictions are then compared to the actual test data, and accuracy metrics are calculated to evaluate the model's performance. Finally, the project generates a comprehensive performance report, documenting the predictive accuracy and providing insights for further refinement, before concluding or preparing to process new data based on user input.

F. New methods and class

CancerPrediction.cs
Lines: 53-70 <code>public static void RunAll (string datasetFileName, string testsetFileName)</code>
Lines: 410-415 <code>private static string EnsureDirectory (string path)</code>
Lines: 421-425 <code>private static int CharToIndex (char c)</code>

HelperMethods.cs
Lines: 39-60 <code>public static HtmConfig FetchEnhancedHTMConfig (int inputBits, int numColumns)</code>
Lines: 67-82 <code>public static EncoderBase GetEnhancedEncoder (int inputBits)</code>

Program.cs
Lines: 166-187 <code>private static string PredictElement (Predictor predictor, int current, int next, ref int matchCount)</code>
Lines: 194-201 <code>private static double CalculateAccuracy (int matchCount, int predictions)</code>

MultiSequenceLearning.cs
Lines: 34-227 <code>private Predictor RunExperiment (int inputBits, HtmConfig cfg, EncoderBase encoder, List<Sequence> sequences)</code>

Table.2: Classes and Methods

In this given table 2 is represented the classes and methods that's changes in our project code. The given lines will be found in the specific class.

RESULTS

In this project, we explored two approaches to multi-sequence learning by repeatedly running experiments across various datasets. To reduce execution time, we deliberately limited the size of each dataset and the number of sequences processed. Accuracy is calculated after testing by "Accuracy = of correct next-step predictions ÷ of predictions attempted × 100". In the project code : " accuracy = (double)matchCount / predictions * 100 ". Where prediction is the number of opportunities to guess the *next* symbol in the test sequence and matchCount is incremented every time the element predicted by the HTM classifier for time equals the true element. We have learned four sequences and we have tested four sequences. We did not take any subsequences. For time consuming, we did not take any subsequences also we used small sequence.

Result for Cancer Prediction

Here the given figure 6 shown the prediction of the next element for the Cancer Prediction scenario. For example, the sequence CDGH predicted the next elements with 100% accuracy.

```
*****#####*****
Test Sequence Test1: EFGH
Input: E
Predicted Sequence: S1 - Predicted next element: F
Input: F
Predicted Sequence: S1 - Predicted next element: G
Input: G
Predicted Sequence: S1 - Predicted next element: H
Accuracy for Test1 sequence: 100%
*****#####*****
*****#####*****
Test Sequence Test2: PQRS
Input: P
Predicted Sequence: S2 - Predicted next element: Q
Input: Q
Predicted Sequence: S2 - Predicted next element: R
Input: R
Predicted Sequence: S2 - Predicted next element: S
Accuracy for Test2 sequence: 100%
*****#####*****
*****#####*****
Test Sequence Test3: ABCD
Input: A
Predicted Sequence: S3 - Predicted next element: B
Input: B
Predicted Sequence: S3 - Predicted next element: C
Input: C
Predicted Sequence: S3 - Predicted next element: D
Accuracy for Test3 sequence: 100%
*****#####*****
*****#####*****
Test Sequence Test4: CDGH
Input: C
Predicted Sequence: S4 - Predicted next element: D
Input: D
Predicted Sequence: S4 - Predicted next element: G
Input: G
Predicted Sequence: S4 - Predicted next element: H
Accuracy for Test4 sequence: 100%
*****#####*****
```

Fig.6: Prediction of next elements with accuracy for Cancer Prediction scenario

Result for Power Consumption Prediction

Figure 7 shows the Prediction of the next elements with accuracy for the Power Consumption scenario. For example, the sequence {3, 4, 7, 8, 9} predicted next elements with 100% accuracy.

```
*****#####*****
Test Sequence Test1: 5, 6, 7, 8
Input: 5
Predicted Sequence: S1_7 - Predicted next element: 6
Input: 6
Predicted Sequence: S1_ - Predicted next element: 7
Input: 7
Predicted Sequence: S1_ - Predicted next element: 8
*****#####*****
Accuracy for Test1 sequence: 100%
*****#####*****
Test Sequence Test2: 6, 11, 12, 13
Input: 6
Predicted Sequence: S2_12 - Predicted next element: 11
Input: 11
Predicted Sequence: S2_13 - Predicted next element: 12
Input: 12
Predicted Sequence: S2_4 - Predicted next element: 13
*****#####*****
Accuracy for Test2 sequence: 100%
*****#####*****
Test Sequence Test3: 1, 2, 3, 4
Input: 1
Predicted Sequence: S3_3 - Predicted next element: 2
Input: 2
Predicted Sequence: S3_4 - Predicted next element: 3
Input: 3
Predicted Sequence: S3_8 - Predicted next element: 4
*****#####*****
Accuracy for Test3 sequence: 100%
*****#####*****
Test Sequence Test4: 3, 4, 7, 8, 10
Input: 3
Predicted Sequence: S4_7 - Predicted next element: 4
Input: 4
Predicted Sequence: S4_8 - Predicted next element: 7
Input: 7
Predicted Sequence: S4_10 - Predicted next element: 8
Input: 8
Predicted Sequence: S4_11 - Predicted next element: 10
*****#####*****
Accuracy for Test4 sequence: 100%
```

Fig.7: Prediction of next elements with accuracy for power consumption scenario

Conclusion

In conclusion, our research emphasizes the importance of Multisequence Learning in enhancing predictive analytics across multiple domains. We demonstrated the potential of a novel strategy for automating sequence extraction and prediction testing from .JSON files to increase efficiency and accuracy when compared to manual approaches. Our findings demonstrate the technique's application across a wide range of businesses, from power consumption forecasting to cancer prediction. Through testing, we proved the effectiveness of our technique in accurately predicting sequences, confirming its value for a wide range of applications. In essence, our study not only explains the principles of Multisequence Learning, but it also pioneers a streamlined way for propelling sequence prediction toward speedier and more reliable outcomes, supporting innovation and development in predictive modeling paradigms.

REFERENCES

- [1] J. H. & S. Blakeslee, On Intelligence. Times Books, 2004.
- [2] J. H. & S. Ahmad, "Why Neurons Have Thousands of Synapses, a Theory of Sequence Memory in Neocortex," vol. 10, 2016.
- [3] J. H. & S. Blakeslee, On Intelligence. New York, Times Books, 2006.
- [4] S. A. & D. D. Jeff Hawkins, HTM Cortical Learning Algorithms., Numenta Inc. Whitepaper, 2011.
- [5] P. Kanerva, Sparse Distributed Memory, Cambridge, Mass. : MIT Press, 1998, p. 155.
- [6] B. G. t. Csongor Pilinszki-Nagy1, "Performance Analysis of Sparse Matrix Representation in Hierarchical Temporal Memory for Sequence Modeling," *INFOCOMMUNICATIONS JOURNAL*, vol. 12, p. 50, 2020.
- [7] P. A. Simona Lodato, "Generating Neuronal Diversity in the Mammalian Cerebral Cortex," *HHS Author Manuscripts*, 2015.
- [8] D. Dobric, "Neocortex," [Online]. Available: <https://github.com/ddobric/neocortexapi>.